Analiza Algoritmilor - A_sol **Nume:**
14-NOV-2025 **Grupă:**

1. Fie următorul fragment de cod:

```
for (int k = n - 100; k <= n + 100; k++)
    for (int i = 1; i <= n; i = i * 2)
        for (int j = 1; j <= n; j += i)
            printf("Hello!\n");
```

Și T(n) funcția care mapează variabila n la numărul de operații elementare realizate (comparații, incrementări, atribuiri etc.). În care dintre următoarele mulțimi se află T(n)?

A. $\Theta(n \log n)$ **B.** $o(n^2)$ **C.** $O(\sqrt{n})$
D. $O(\log n)$ **E.** $\Omega(\log n)$ **F.** $\Theta(n)$

2. Care din următoarele funcții se află în $o(n)$:

A. $w(n) = 4n + 2$ **B.** $u(n) = \sqrt{n}$ **C.** $v(n) = n \log n$
D. $r(n) = 2n$ **E.** $s(n) = \log n$ **F.** $t(n) = 8$

3. Presupunem un `ArrayList` care, la umplere, își triplează capacitatea. Pornim de la capacitate = 1 și număr elemente = 0. Care este costul mediu pentru introducerea a 80 de elemente? (costul inserției = 1, costul copierii unui element = 1)

A. $c \approx 2$ **B.** $c \approx 2.5$ **C.** $c \approx 1.5$
D. $c \approx 3$ **E.** $c \approx 3.5$ **F.** $c \approx 1$

4. Care din următoarele operații au cost constant $O(1)$:

A. acces la poziție arbitrară k în `ArrayList` **B.** inserare la poziție arbitrară k în `ArrayList` **C.** acces la poziție arbitrară k în `LinkedList`
D. inserare la sfârșit în `LinkedList` **E.** inserare la sfârșit în `ArrayList` **F.** inserare la poziție arbitrară k în `LinkedList`

5. Care recurențe pot fi rezolvate cu Teorema Master?

A. $T(n) = T(\sqrt{n}/2) + sqrt(n)$ **B.** $T(n) = T(\sqrt{n}) + 1$ **C.** $T(n) = 2T(n/2) + n$
D. $T(n) = 4T(n/4) + n^3$ **E.** $T(n) = T(n-1) + 1$ **F.** $T(n) = T(n/2) + n/2$

6. Ce relație de recurență caracterizează codul:

```
int max(const int *a, int lo, int hi) {
    if (lo == hi) return a[lo];
    int mid = (lo + hi) / 2;
    int left  = max(a, lo, mid);
    int right = max(a, mid + 1, hi);
    return (left > right) ? left : right;
}
```

A. $T(n) = 2T(n-1) + \Theta(1)$ **B.** $T(n) = 2T(n-1) + \Theta(n)$ **C.** $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$
D. $T(n) = T\left(\frac{n}{2}\right) + \Theta(n)$ **E.** $T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$ **F.** $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(1)$

7. Într-un turneu, la fiecare rundă sunt evaluați toți jucătorii rămași, apoi jumătate sunt eliminați. Costul evaluării este proporțional cu numărul de jucători. Dacă la început sunt 2^n jucători, care este timpul total de evaluare?

A. $\Theta(n2^n)$ **B.** $\Theta(2^n \log n)$ **C.** $\Theta(\log n)$
D. $\Theta(n)$ **E.** $\Theta(2^n)$ **F.** $\Theta(n \log n)$

8. Alegeți combinația corectă de axiome pentru operația $\text{isSorted} : \text{List} \to \text{Boolean}$

A. $\text{isSorted}(\text{Void}) = \text{false}$ **B.** $\text{isSorted}(\text{Void}) = \text{true}$ **C.** $\text{isSorted}(\text{Cons}(x, \text{Void})) = \text{true}$
D. $\text{isSorted}(\text{Cons}(x, \text{Void})) = \text{false}$ **E.** $\text{isSorted}(\text{Cons}(x, \text{Cons}(y, xs))) = x \leq y \;\&\&\; \text{isSorted}(xs)$ **F.** $\text{isSorted}(\text{Cons}(x, \text{Cons}(y, xs))) = x \leq y \;\&\&\; \text{isSorted}(\text{Cons}(y, xs))$

9. Într-un array care se dublează când e plin și se înjumătățește când e 1/4 plin:

A. inserarea unui element are cost amortizat $O(1)$
B. eliminarea ultimului element are cost amortizat $O(n)$
C. eliminarea ultimului element are cost amortizat $O(1)$
D. inserarea unui element are cost amortizat $O(n)$

10. Fie un TDA `BTree` cu constructorii: Nil : BTree; Node : $\mathbb{Z} \times$ BTree $\times$ BTree $\to$ BTree
Completați axiomele operatorului care oglindește un arbore:

$\text{mirror}(\text{Nil}) =$ Nil

$\text{mirror}(\text{Node}(x, l, r)) =$ $\text{Node}(x, \text{mirror}(r), \text{mirror}(l))$

11. Fie un ArrayList care are inițial capacitate 4 și 0 elemente; de câte ori se va dubla când inserăm 20 de elemente?

3

12. Folosiți o notație asimptotică pentru a descrie următoarea mulțime de funcții: $\frac{\omega(n^3)}{\Theta(n)}$

$\omega(n^2)$

13. Dați un exemplu de funcție din $\Omega(n)$ care nu este în $\omega(n)$.

$f(n) = n$

14. Ce complexitate (notație Θ) are recurența: $T(n) = 4T(n/4) + n$

$\Theta(n \log n)$

15. Scrieți cazul de bază pentru a demonstra prin inducție structurală proprietatea: $\text{size}(l) = \text{size}(\text{reverse}(l))$

$\text{size}(\texttt{Empty}) \overset{REV1}{=} \text{size}(\text{reverse}(\texttt{Empty}))$